

SPRING BOOT PRACTICO

JPA + SQLite - Banking Lab

Aprende como en un libro clasico: ejemplo -> prueba -> ejercicio

Proyecto: API bancaria con clientes, cuentas, movimientos y transferencias

Edicion 2 - Octubre de 2026

0. Como usar este libro

La regla es simple: escribe codigo, ejecuta, rompe, entiende y vuelve a intentar.

Esta guia no asume que ya sabes crear un Controller, un Repository o un Service. Cada concepto aparece primero en un ejemplo minimo que puedes ejecutar. Despues haces una variacion guiada y finalmente un ejercicio donde debes pensar la logica.

Metodo de estudio: No copies el ejercicio desde una solucion externa. Si te bloqueas, vuelve al ejemplo del mismo capitulo e identifica que parte cambia y que parte se mantiene.

- Ejemplo: codigo completo y pequeno para entender una sola idea.
- Prueba: una peticion concreta que debes ejecutar y observar.
- Variacion guiada: cambias una parte pequena usando la misma estructura.
- Ejercicio: implementas una necesidad distinta reutilizando la base aprendida.
- Checkpoint: una peticion de Postman confirma si el capitulo quedo bien.

El proyecto ya trae un DemoController funcional, una base SQLite con datos y una coleccion de Postman. Los paquetes de negocio estan vacios a proposito.

1. El mapa mental de Spring Boot

Antes de memorizar anotaciones, entiende por donde viaja una peticion.

```
Postman / navegador
|
v
Controller -> recibe HTTP
|
v
Service -> decide la logica
|
v
Repository -> consulta o guarda
|
v
SQLite
```

Un Controller no deberia decidir si una cuenta tiene saldo suficiente. Un Repository no deberia decidir si un cliente puede abrir una cuenta. Cada capa tiene una responsabilidad.

Capa	Responsabilidad	Ejemplo
Controller	HTTP: URL, body, status	GET /api/clientes/1
Service	Reglas de negocio	Buscar cliente o lanzar error
Repository	Acceso a datos	findById(1)
Entity	Mapa Java <-> tabla	Cliente <-> clientes
DTO	Datos de entrada/salida	ClienteCreateRequest

Objetivo final: Al terminar debes poder abrir un proyecto Spring Boot y seguir el recorrido Controller -> Service -> Repository -> base de datos sin sentir que las cosas ocurren por magia.

2. Levanta el proyecto antes de tocar nada

1. Descomprime springboot-jpa-sqlite-lab-v2.zip.
2. Abre la carpeta en IntelliJ como proyecto Maven.
3. Espera a que Maven resuelva las dependencias.
4. Ejecuta BankingLabApplication.
5. Importa Banking_Lab.postman_collection.json en Postman.
6. Ejecuta la carpeta 00 - Demo resuelto.

El proyecto usa Spring Boot 3.5.16, Java 17, Spring Web, Spring Data JPA, Validation y SQLite. La base no se crea al arrancar: ya existe.

```
spring.datasource.url=jdbc:sqlite:banking-lab.db  
spring.jpa.database-platform=org.hibernate.community.dialect.SQLiteDialect  
spring.jpa.hibernate.ddl-auto=none
```

Por que ddl-auto=none: Hibernate no debe inventar ni modificar tablas durante el curso. Tu trabajo es mapear correctamente el esquema que ya existe.

3. Conoce la base de datos que ya tienes

La base banking-lab.db viene poblada. Esto permite practicar lecturas desde JPA desde el primer momento sin perder tiempo creando datos a mano.

3.1 Tabla clientes

id	documento	nombre_completo	correo	activo
1	100000001	Jhoan Hernandez	jhoan@bankinglab.local	1
2	100000002	Laura Gomez	laura@bankinglab.local	1
3	100000003	Mateo Torres	mateo@bankinglab.local	1
4	100000004	Camila Ruiz	camila@bankinglab.local	1
5	100000005	Andres Pena	andres@bankinglab.local	0

3.2 Tabla cuentas

id	numero_cuenta	cliente_id	tipo	saldo	activa
1	0010000001	1	AHORROS	1,500,000	1
2	0010000002	1	CORRIENTE	350,000	1
3	0020000001	2	AHORROS	2,800,000	1
4	0030000001	3	AHORROS	75,000	1
5	0040000001	4	AHORROS	4,200,000	1
6	0050000001	5	CORRIENTE	900,000	0
7	0020000002	2	AHORROS	125,000	1

3.3 Tabla movimientos

Hay 13 movimientos iniciales con tipos DEPOSITO, RETIRO, TRANSFERENCIA_ENTRADA y TRANSFERENCIA_SALIDA.

clientes 1 ---- N cuentas 1 ---- N movimientos

Reset: Si alteras saldos o datos y quieres volver al inicio, ejecuta: `python database/reset_database.py`

4. Tu primer Controller: que es y como se crea

Un Controller es una clase que recibe peticiones HTTP. Spring la detecta porque tiene `@RestController`. `@RequestMapping` define el prefijo comun de sus rutas.

4.1 Ejemplo completo

```
package com.jhoan.bankinglab.controller;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import java.util.Map;

@RestController
@RequestMapping("/api/demo")
public class DemoController {

    @GetMapping("/saludo/{nombre}")
    public Map<String, String> saludar(@PathVariable String nombre) {
        return Map.of("mensaje", "Hola " + nombre);
    }
}
```

4.2 Lee el codigo como una frase

- `@RestController`: esta clase atiende peticiones HTTP y devuelve datos.
- `@RequestMapping("/api/demo")`: todas sus rutas empiezan con `/api/demo`.
- `@GetMapping("/saludo/{nombre}")`: este metodo responde a GET.
- `@PathVariable`: toma un valor que viene dentro de la URL.
- El Map se convierte automaticamente a JSON.

4.3 Prueba

```
GET http://localhost:8080/api/demo/saludo/Jhoan

{
  "mensaje": "Hola Jhoan"
}
```

4.4 Variacion guiada

Agrega otro metodo dentro de `DemoController`. La estructura del GET es la misma; cambia la logica.

```
@GetMapping("/doble/{numero}")
public Map<String, Integer> doble(@PathVariable int numero) {
    return Map.of("resultado", numero * 2);
}
```

4.5 Ejercicio 1

Tu turno: Crea GET `/api/demo/cuadrado/{numero}`. Debe responder `{"resultado": 49}` cuando `numero` sea 7.

5. GET con parametros: PathVariable vs RequestParam

Un PathVariable identifica algo dentro de la ruta. Un RequestParam viaja despues del signo ?. Ambos terminan siendo parametros Java.

5.1 Ejemplo con RequestParam

```
@GetMapping("/suma")
public Map<String, Integer> sumar(
    @RequestParam int a,
    @RequestParam int b) {
    return Map.of("resultado", a + b);
}

GET /api/demo/suma?a=7&b=5
```

5.2 Cuando usar cada uno

Necesidad	Forma	Ejemplo
Identificar recurso	PathVariable	/clientes/15
Filtrar	RequestParam	/clientes?activo=true
Paginar	RequestParam	/movimientos?page=0&size=10

5.3 Ejercicio 2

Tu turno: Crea GET /api/demo/edad-futura?edad=20&anios=5. La respuesta debe ser {"edadFutura": 25}.

Piensa que parte del metodo sumar puedes reutilizar casi sin cambiar.

6. JPA Entity: convertir una tabla en una clase Java

JPA necesita una clase que describa como se corresponde cada registro SQL con un objeto Java. Esa clase es una Entity.

6.1 Ejemplo: Cliente

```
package com.jhoan.bankinglab.entity;

import jakarta.persistence.*;
import java.time.LocalDateTime;

@Entity
@Table(name = "clientes")
public class Cliente {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true)
    private String documento;

    @Column(name = "nombre_completo", nullable = false)
    private String nombreCompleto;

    @Column(nullable = false, unique = true)
    private String correo;

    @Column(nullable = false)
    private Boolean activo;

    @Column(name = "fecha_creacion", nullable = false)
    private LocalDateTime fechaCreacion;

    public Cliente() {}

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public String getDocumento() { return documento; }
    public void setDocumento(String documento) { this.documento = documento; }
    public String getNombreCompleto() { return nombreCompleto; }
    public void setNombreCompleto(String nombreCompleto) { this.nombreCompleto = nombreCompleto; }
    public String getCorreo() { return correo; }
    public void setCorreo(String correo) { this.correo = correo; }
    public Boolean getActivo() { return activo; }
    public void setActivo(Boolean activo) { this.activo = activo; }
    public LocalDateTime getFechaCreacion() { return fechaCreacion; }
    public void setFechaCreacion(LocalDateTime fechaCreacion) { this.fechaCreacion = fechaCreacion; }
}
```

6.2 Lo importante

- @Entity hace que Hibernate administre la clase.
- @Table conecta la clase con la tabla clientes.
- @Id identifica la PK.
- @Column(name = "nombre_completo") conecta camelCase con la columna SQL.
- JPA necesita constructor sin argumentos.

Hazlo ahora: Crea Cliente.java dentro de entity. Puedes generar getters/setters desde IntelliJ; no necesitas escribirlos a mano si no quieres.

6.3 Ejercicio 3

Sin copiar una solución completa, crea la Entity Cuenta usando este esquema:

```
cuentas
- id INTEGER PK
- numero_cuenta TEXT UNIQUE
- cliente_id INTEGER FK
- tipo TEXT
- saldo NUMERIC
- activa INTEGER
- fecha_creacion TEXT
```

Pista: Por ahora no mapees cliente_id como relación. Usa un campo Long clienteId con @Column(name = "cliente_id"). La relación @ManyToOne llega después.

7. Repository: hablar con SQLite sin escribir SQL

JpaRepository ya trae operaciones como findAll, findById, save y delete. Tu interfaz solo declara lo que necesita.

7.1 Ejemplo

```
package com.jhoan.bankinglab.repository;

import com.jhoan.bankinglab.entity.Cliente;
import org.springframework.data.jpa.repository.JpaRepository;

public interface ClienteRepository
    extends JpaRepository<Cliente, Long> {
}
```

Con esas cuatro líneas ya puedes ejecutar clienteRepository.findAll() o clienteRepository.findById(1L). Spring crea la implementación en tiempo de ejecución.

7.2 Query Methods

```
import java.util.List;
import java.util.Optional;

Optional<Cliente> findByDocumento(String documento);
List<Cliente> findByActivo(Boolean activo);
boolean existsByCorreo(String correo);
```

Spring interpreta el nombre del método y construye la consulta. No es magia: es una convención de nombres.

7.3 Ejercicio 4

Tu turno: Crea CuentaRepository. Debe extender JpaRepository<Cuenta, Long> y tener un método para buscar por numeroCuenta.

Pista: Si el atributo Java se llama numeroCuenta, el método empieza por findByNumeroCuenta.

8. Service: donde vive la logica

El Service usa el Repository y decide que hacer con el resultado. Aqui empiezan las reglas de negocio.

8.1 Ejemplo

```
package com.jhoan.bankinglab.service;

import com.jhoan.bankinglab.entity.Cliente;
import com.jhoan.bankinglab.repository.ClienteRepository;
import org.springframework.stereotype.Service;

import java.util.List;

@Service
public class ClienteService {

    private final ClienteRepository repository;

    public ClienteService(ClienteRepository repository) {
        this.repository = repository;
    }

    public List<Cliente> listar() {
        return repository.findAll();
    }

    public Cliente buscarPorId(Long id) {
        return repository.findById(id)
            .orElseThrow() ->
            new RuntimeException("Cliente no encontrado");
    }
}
```

8.2 Que esta pasando con el constructor

Tu clase no hace new ClienteRepository(). Spring crea el Repository y lo entrega al constructor. Eso es inyeccion de dependencias.

8.3 Ejercicio 5

Tu turno: Agrega buscarPorDocumento(String documento) en ClienteService usando el Query Method del capitulo anterior. Si no existe, lanza RuntimeException por ahora.

9. Primer endpoint real contra SQLite

Ahora conectas Controller -> Service -> Repository -> SQLite. Este es el punto donde las piezas empiezan a tener sentido juntas.

9.1 Ejemplo ClienteController

```
package com.jhoan.bankinglab.controller;

import com.jhoan.bankinglab.entity.Cliente;
import com.jhoan.bankinglab.service.ClienteService;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/clientes")
public class ClienteController {

    private final ClienteService service;

    public ClienteController(ClienteService service) {
        this.service = service;
    }

    @GetMapping
    public List<Cliente> listar() {
        return service.listar();
    }

    @GetMapping("/{id}")
    public Cliente buscarPorId(@PathVariable Long id) {
        return service.buscarPorId(id);
    }
}
```

9.2 Checkpoint Postman

```
GET /api/clientes
GET /api/clientes/1
```

El primer GET debe devolver 5 clientes. El segundo debe devolver a Jhoan Hernandez.

9.3 Ejercicio 6

Tu turno: Implementa GET /api/clientes/documento/{documento}. Debe funcionar con 100000002 y devolver a Laura Gomez.

Observa que el Controller cambia muy poco. La diferencia real esta en el metodo de Service que invocas.

10. DTO: no expongas la Entity porque si

Devolver Entity directamente sirve para aprender, pero en una API real conviene controlar el contrato. Un DTO decide exactamente que entra y que sale.

10.1 Ejemplo con record

```
package com.jhoan.bankinglab.dto;

public record ClienteResponse(
    Long id,
    String documento,
    String nombreCompleto,
    String correo,
    Boolean activo
) {}

private ClienteResponse toResponse(Cliente cliente) {
    return new ClienteResponse(
        cliente.getId(),
        cliente.getDocumento(),
        cliente.getNombreCompleto(),
        cliente.getCorreo(),
        cliente.getActivo()
    );
}
```

Record: Un record es una forma compacta de representar datos inmutables. Para DTOs simples evita mucho código repetitivo.

10.2 Ejercicio 7

Tu turno: Refactoriza `listar` y `buscarPorId` para devolver `ClienteResponse` en lugar de `Cliente`. El JSON debe conservar los mismos datos visibles, excepto `fechaCreacion` si decides no exponerla.

11. POST: recibir JSON, validar y guardar

POST introduce `@RequestBody`. Spring convierte el JSON entrante en un objeto Java. Con `@Valid` puedes validar antes de entrar a la logica.

11.1 DTO de entrada

```
package com.jhoan.bankinglab.dto;

import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;

public record ClienteCreateRequest(
    @NotBlank String documento,
    @NotBlank String nombreCompleto,
    @NotBlank @Email String correo
) {}
```

11.2 Service para crear

```
public ClienteResponse crear(ClienteCreateRequest request) {
    Cliente cliente = new Cliente();
    cliente.setDocumento(request.documento());
    cliente.setNombreCompleto(request.nombreCompleto());
    cliente.setCorreo(request.correo());
    cliente.setActivo(true);
    cliente.setFechaCreacion(LocalDate.now());

    Cliente guardado = repository.save(cliente);
    return toResponse(guardado);
}
```

11.3 Controller

```
@PostMapping
public ResponseEntity<ClienteResponse> crear(
    @Valid @RequestBody ClienteCreateRequest request) {
    ClienteResponse creado = service.crear(request);
    return ResponseEntity.status(201).body(creado);
}
```

11.4 Ejercicio 8

Tu turno: Antes de guardar, evita documentos y correos duplicados usando `existsByDocumento` y `existsByCorreo`. Por ahora puedes lanzar `RuntimeException`.

Prueba Crear cliente en la carpeta 02 de Postman. La coleccion guardara el id creado en una variable.

12. PUT y DELETE: modificar estado sin borrar historia

En un sistema financiero suele ser mas razonable desactivar que borrar fisicamente. Usaremos DELETE como operacion HTTP, pero internamente cambiaremos activo a false.

12.1 DTO de actualizacion

```
public record ClienteUpdateRequest(  
    @NotBlank String nombreCompleto,  
    @NotBlank @Email String correo  
) {}
```

12.2 Ejemplo PUT

```
@PutMapping("/{id}")  
public ClienteResponse actualizar(  
    @PathVariable Long id,  
    @Valid @RequestBody ClienteUpdateRequest request) {  
    return service.actualizar(id, request);  
}
```

12.3 Ejemplo DELETE logico

```
@DeleteMapping("/{id}")  
public ResponseEntity<Void> desactivar(@PathVariable Long id) {  
    service.desactivar(id);  
    return ResponseEntity.noContent().build();  
}
```

12.4 Ejercicio 9

Tu turno: Implementa actualizar y desactivar en ClienteService. Para desactivar: busca el cliente, setActivo(false) y save.

13. Relaciones JPA: Cuenta pertenece a Cliente

Antes modelaste clienteId como Long para concentrarte en lo basico. Ahora reemplaza ese campo por una relacion real.

13.1 Ejemplo @ManyToOne

```
@ManyToOne(fetch = FetchType.LAZY, optional = false)
@JoinColumn(name = "cliente_id", nullable = false)
private Cliente cliente;
```

Muchas cuentas pueden pertenecer a un cliente. No necesitas crear una lista bidireccional en Cliente para resolver el laboratorio.

13.2 CuentaResponse

```
public record CuentaResponse(
    Long id,
    String numeroCuenta,
    Long clienteId,
    String tipo,
    BigDecimal saldo,
    Boolean activa
) {}
```

13.3 Repository

```
Optional<Cuenta> findByNumeroCuenta(String numeroCuenta);
List<Cuenta> findByClienteId(Long clienteId);
```

13.4 Ejercicio 10

Tu turno: Implementa CuentaService y CuentaController con GET /api/cuentas, GET /api/cuentas/{id}, GET /api/cuentas/numero/{numero} y GET /api/clientes/{id}/cuentas.

Pista: En el DTO devuelve cliente.getId(), no el objeto Cliente completo.

14. Abrir una cuenta: POST con una relacion

14.1 Request

```
public record CuentaCreateRequest(  
    Long clienteId,  
    String tipo,  
    BigDecimal saldoInicial  
) {}
```

Para crear una Cuenta no basta con copiar clienteId. Primero debes buscar el Cliente y asignar el objeto encontrado a `cuenta.setCliente(cliente)`.

14.2 Secuencia

```
clienteRepository.findById(request.clienteId())  
-> validar cliente activo  
-> generar numero de cuenta  
-> crear Cuenta  
-> repository.save(cuenta)  
-> devolver CuentaResponse
```

14.3 Ejercicio 11

Tu turno: Implementa POST `/api/cuentas`. Reglas: cliente debe existir, debe estar activo, tipo solo AHORROS o CORRIENTE, saldoInicial no puede ser negativo y la nueva cuenta queda activa.

Generacion simple: Puedes formar `numeroCuenta` con `System.currentTimeMillis()` mientras aprendes. No estamos diseñando el core bancario nacional todavía.

15. Depositos: la primera regla de negocio real

Un deposito ya no es CRUD simple. Hay que validar monto, modificar saldo y luego persistir.

15.1 DTO

```
public record OperacionRequest(
    @DecimalMin(value = "0.01") BigDecimal monto,
    @NotBlank String descripcion
) {}
```

15.2 Ejemplo de logica

```
public CuentaResponse depositar(
    Long cuentaId,
    OperacionRequest request) {

    Cuenta cuenta = buscarEntidad(cuentaId);

    if (!cuenta.getActiva()) {
        throw new RuntimeException("Cuenta inactiva");
    }

    cuenta.setSaldo(cuenta.getSaldo().add(request.monto()));
    Cuenta guardada = repository.save(cuenta);
    return toResponse(guardada);
}
```

15.3 Endpoint

```
@PostMapping("/{id}/depositos")
public CuentaResponse depositar(
    @PathVariable Long id,
    @Valid @RequestBody OperacionRequest request) {
    return service.depositar(id, request);
}
```

15.4 Ejercicio 12

Tu turno: Implementa el deposito y prueba la cuenta 1. Antes vale 1,500,000. Si depositas 50,000 debe quedar en 1,550,000.

16. Movimientos: cada operacion debe dejar rastro

Modificar el saldo sin registrar un movimiento hace que el sistema pierda trazabilidad. Ahora crearas una Entity Movimiento y guardaras uno por cada operacion.

16.1 Entity esencial

```
@Entity
@Table(name = "movimientos")
public class Movimiento {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "cuenta_id")
    private Cuenta cuenta;

    private String tipo;
    private BigDecimal monto;
    private String descripcion;
    private String referencia;

    @Column(name = "fecha_creacion")
    private LocalDateTime fechaCreacion;

    public Movimiento() {}
}
```

Genera getters/setters en IntelliJ. Crea MovimientoRepository extends JpaRepository<Movimiento, Long>.

16.2 Ejercicio 13

Tu turno: Despues de depositar, guarda un movimiento DEPOSITO con el mismo monto y descripcion. Implementa GET /api/cuentas/{id}/movimientos ordenado del mas reciente al mas antiguo.

Pista Repository: Prueba un metodo findByCuentaIdOrderByFechaCreacionDesc(Long cuentaId).

17. Retiros: validar saldo antes de tocarlo

17.1 Regla

```
if (cuenta.getSaldo().compareTo(request.monto()) < 0) {  
    throw new RuntimeException("Saldo insuficiente");  
}  
  
cuenta.setSaldo(cuenta.getSaldo().subtract(request.monto()));
```

BigDecimal no se compara con < o >. compareTo devuelve negativo, cero o positivo.

17.2 Ejercicio 14

Tu turno: Implementa POST /api/cuentas/{id}/retiros. Valida cuenta activa, monto positivo y saldo suficiente. Registra un movimiento RETIRO.

Postman trae dos casos: un retiro valido y otro absurdo sobre la cuenta 4, que solo tiene 75,000.

18. Transferencias y @Transactional

Una transferencia modifica al menos dos saldos y crea dos movimientos. Si algo falla a mitad del proceso, no puedes dejar un debito sin su credito.

18.1 DTO

```
public record TransferenciaRequest(  
    Long cuentaOrigenId,  
    Long cuentaDestinoId,  
    @DecimalMin("0.01") BigDecimal monto,  
    @NotBlank String descripcion  
) {}
```

18.2 Estructura del Service

```
@Transactional  
public void transferir(TransferenciaRequest request) {  
    Cuenta origen = buscarEntidad(request.cuentaOrigenId());  
    Cuenta destino = buscarEntidad(request.cuentaDestinoId());  
  
    validarTransferencia(origen, destino, request.monto());  
  
    origen.setSaldo(origen.getSaldo().subtract(request.monto()));  
    destino.setSaldo(destino.getSaldo().add(request.monto()));  
  
    cuentaRepository.save(origen);  
    cuentaRepository.save(destino);  
  
    guardarMovimiento(origen, "TRANSFERENCIA_SALIDA", request);  
    guardarMovimiento(destino, "TRANSFERENCIA_ENTRADA", request);  
}
```

18.3 Ejercicio 15

Tu turno: Completa validarTransferencia y guardarMovimiento. Reglas: cuentas distintas, ambas activas, monto positivo, saldo suficiente. Usa la misma referencia para los dos movimientos.

Idea clave: @Transactional hace que todas las operaciones formen una unidad. Si una excepcion escapa del metodo, los cambios de esa transaccion pueden revertirse.

19. Excepciones globales: deja de devolver errores horribles

RuntimeException te ayudo mientras aprendias el flujo. Ahora crea excepciones con significado y un handler central.

19.1 Excepcion

```
public class RecursoNoEncontradoException
    extends RuntimeException {

    public RecursoNoEncontradoException(String mensaje) {
        super(mensaje);
    }
}
```

19.2 Handler

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(RecursoNoEncontradoException.class)
    public ResponseEntity<Map<String, Object>> handleNotFound(
        RecursoNoEncontradoException ex) {

        return ResponseEntity.status(404).body(Map.of(
            "status", 404,
            "error", "Not Found",
            "message", ex.getMessage()
        ));
    }
}
```

19.3 Ejercicio 16

Tu turno: Crea una excepcion para reglas de negocio y manejala como 409 Conflict. Maneja tambien MethodArgumentNotValidException como 400 Bad Request.

Prueba Cliente inexistente y Cliente invalido en la carpeta 07 de Postman.

20. Query Methods, filtros y paginacion

20.1 Filtro opcional

```
@GetMapping
public List<ClienteResponse> listar(
    @RequestParam(required = false) Boolean activo) {

    if (activo == null) {
        return service.listar();
    }

    return service.listarPorEstado(activo);
}
```

20.2 Paginacion

```
Page<Movimiento> findByCuentaId(
    Long cuentaId,
    Pageable pageable);

GET /api/movimientos?cuentaId=1&page=0&size=5&sort=fechaCreacion,desc
```

20.3 Ejercicio 17

Tu turno: Implementa filtro activo en clientes y movimientos paginados por cuenta. No devuelvas todos los movimientos del sistema en una sola respuesta.

21. Pruebas: confirma la logica sin depender de Postman

Postman valida el sistema desde afuera. Las pruebas unitarias te permiten comprobar reglas del Service de forma aislada.

21.1 Ejemplo de idea

```
@Test
void retirarDebeFallarSiNoHaySaldo() {
    Cuenta cuenta = new Cuenta();
    cuenta.setSaldo(new BigDecimal("1000"));
    cuenta.setActiva(true);

    OperacionRequest request = new OperacionRequest(
        new BigDecimal("1500"),
        "prueba"
    );

    assertThrows(
        ReglaNegocioException.class,
        () -> service.retirar(1L, request)
    );
}
```

Para que esta prueba sea realmente unitaria necesitaras mockear repositories con Mockito. No es obligatorio para terminar el laboratorio, pero es el siguiente paso natural.

21.2 Ejercicio 18

Tu turno: Escribe al menos tres pruebas: deposito suma saldo, retiro rechaza saldo insuficiente y transferencia rechaza misma cuenta origen/destino.

22. Reto final: construye una funcionalidad sin receta

Ya no hay ejemplo equivalente exacto. Debes combinar lo aprendido.

Historia: Como cliente quiero consultar un resumen de una cuenta para ver saldo, cantidad de movimientos y total depositado/retiro en un rango de fechas.

Implementa:

```
GET /api/cuentas/{id}/resumen?desde=2026-09-01&hasta=2026-09-30
```

Respuesta sugerida:

```
{
  "cuentaId": 1,
  "numeroCuenta": "0010000001",
  "saldoActual": 1500000,
  "cantidadMovimientos": 3,
  "totalDepositos": 1700000,
  "totalRetiros": 200000
}
```

- Crea un DTO de respuesta.
- Consulta los movimientos de la cuenta.
- Filtra por fechas.
- Suma depositos y retiros con BigDecimal.
- No coloques la logica en el Controller.

Criterio de exito: Si puedes implementar este endpoint sin buscar una solucion exacta, ya entendiste las piezas fundamentales del backend Spring Boot que este laboratorio pretende enseñar.

23. Checklist de dominio

Concepto	Debes poder explicar
@RestController	Por que una clase recibe HTTP
@GetMapping/@PostMapping	Como se conecta metodo HTTP con metodo Java
@PathVariable	Como leer una parte de la URL
@RequestParam	Como leer filtros/query params
@RequestBody	Como JSON se convierte a Java
@Entity	Como una clase representa una tabla
JpaRepository	Que operaciones te entrega sin implementarlas
@Service	Por que la logica no va en Controller
DTO	Por que no exponer Entity indiscriminadamente
@Valid	Como validar entrada antes de la logica
@ManyToOne	Como representar una FK
BigDecimal	Por que usarlo para dinero
@Transactional	Que problema resuelve una transaccion
@RestControllerAdvice	Como centralizar errores
Pageable	Como evitar respuestas gigantes

No memorices cada anotacion. Si entiendes el flujo y sabes reconocer donde buscar cada responsabilidad, el resto se vuelve repeticion.

24. Mapa de la coleccion de Postman

Carpeta	Cuando usarla
00 - Demo resuelto	Antes de empezar
01 - Clientes GET	Despues del capitulo 9
02 - Clientes escritura	Despues de capitulos 11 y 12
03 - Cuentas	Despues de capitulos 13 y 14
04 - Depositos y retiros	Despues de capitulos 15 a 17
05 - Transferencias	Despues del capitulo 18
06 - Consultas JPA	Despues del capitulo 20
07 - Errores y validaciones	Despues del capitulo 19

Importante: Que una peticion de Postman falle antes de su capitulo no significa que el proyecto este roto. Significa que todavia no has implementado ese endpoint.